

Get data from the internet

1. Before you begin
2. App overview
3. Explore the MarsPhotos starter app
4. App Overview
5. Web services and Retrofit
6. Connecting to the Internet
7. Add Internet permission and Exception Handling
8. Parse the JSON response with Moshi
9. Solution code
10. Summary
11. Learn more

1. Before you begin

Most of the Android apps in the market connect to the Internet to perform some network operations. Such as retrieving emails, messages or similar information from a backend server. Gmail, YouTube, and Google Photos are some example apps that connect to the internet to display the user data.

In this codelab, you use open source developed libraries to build the network layer and get data from a backend server. This greatly simplifies fetching the data, and also helps the app conform to Android best practices, such as performing the operations on a background thread. You will also update the app's user interface if the internet is slow or unavailable; this will keep the user informed about any network connectivity issues.

What you should already know

- How to create and use fragments.
- How to use Android architecture components `ViewModel`, and `LiveData`.
- How to add dependencies in a gradle file.

What you'll learn

- What a [REST](#) web service is.
- Using the [Retrofit](#) library to connect to a REST web service on the internet and get a response.
- Using the [Moshi](#) library to parse the JSON response into a data object.

What you'll do

- Modify a starter app to make a web service API request and handle the response.
- Implement a network layer for your app using the Retrofit library.
- Parse the JSON response from the web service into your app's `LiveData` objects with the Moshi library.
- Use Retrofit's support for coroutines to simplify the code.

What you need

- A computer with Android Studio installed.
- Starter code for the MarsPhotos app.

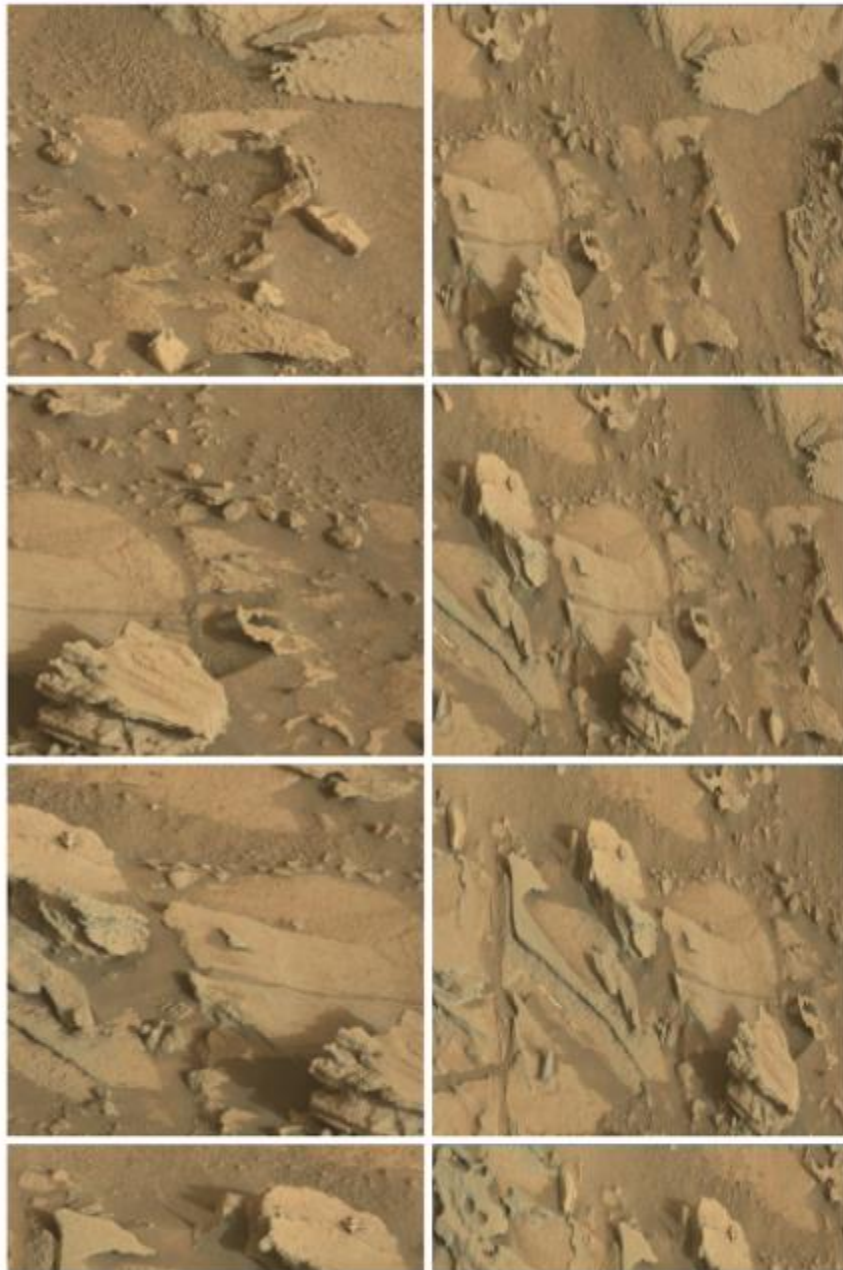
2. App overview

In this pathway, you work with a starter app called **MarsPhotos**, which shows images of Mars surface. This app connects to a web service to retrieve and display the Mars photos. The images are real-life photos from Mars captured from NASA's Mars rovers. Following is the screenshot of the final app, which contains a grid of thumbnail property images, built with a `RecyclerView`.

11:52

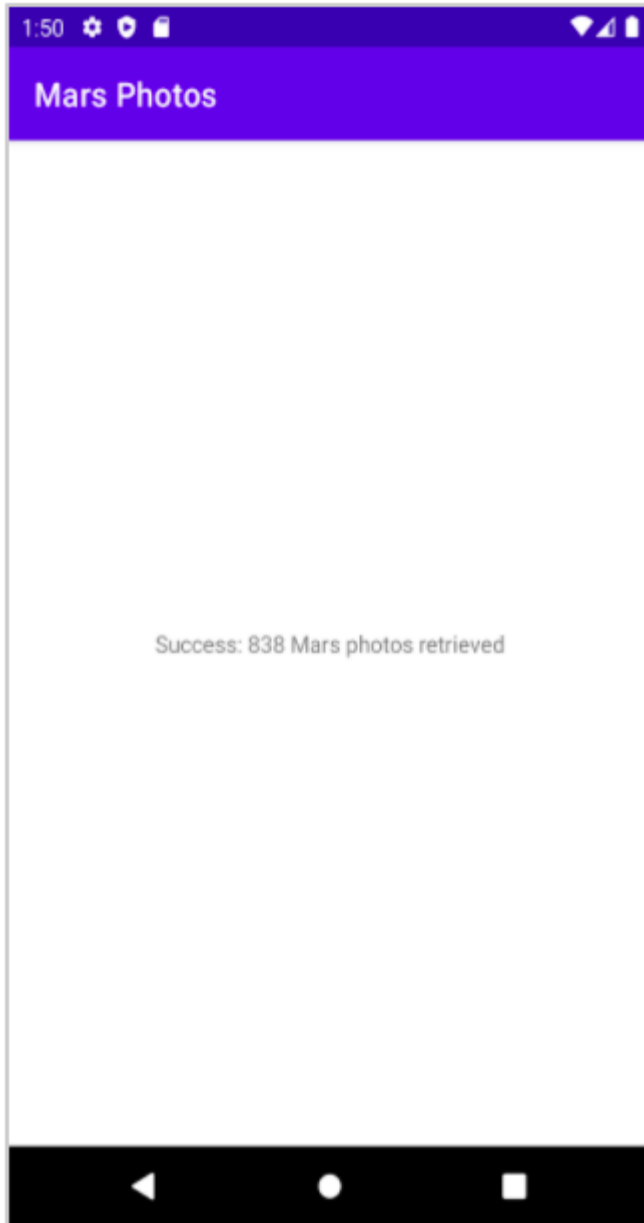


Mars Photos



Note: The above screenshot is the screenshot of the final app that you will build at the end of the pathway that is at the end of the next codelab. This screenshot is shown in this codelab to give you a better idea of the overall app functioning.

The version of the app you build in this codelab won't have a lot of visual flash: it focuses on the networking layer part of the app to connect to the internet and download the raw property data using a web service. To ensure that the data is correctly retrieved and parsed, you'll just print the number of photos received from the backend server in a text view:



3. Explore the MarsPhotos starter app

Download starter code

This codelab provides starter code for you to extend with features taught in this codelab. Starter code may contain code that is both familiar and unfamiliar to you from previous codelabs. You will learn more about unfamiliar code in later codelabs.

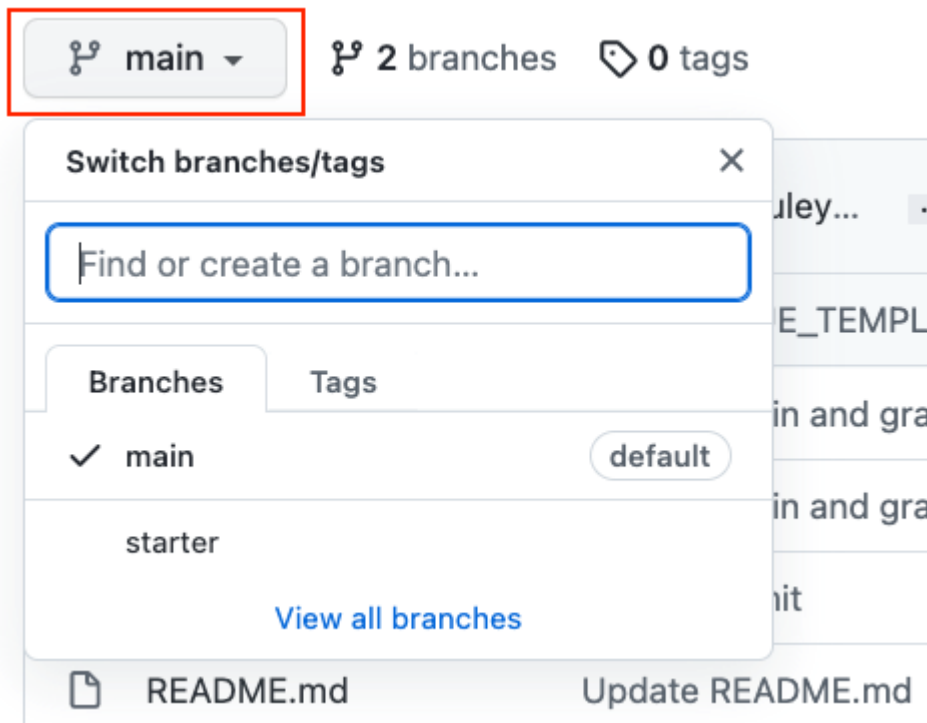
If you use the starter code from GitHub, note that the folder name is `android-basics-kotlin-mars-photos-app`. Select this folder when you open the project in Android Studio.

Starter Code URL:

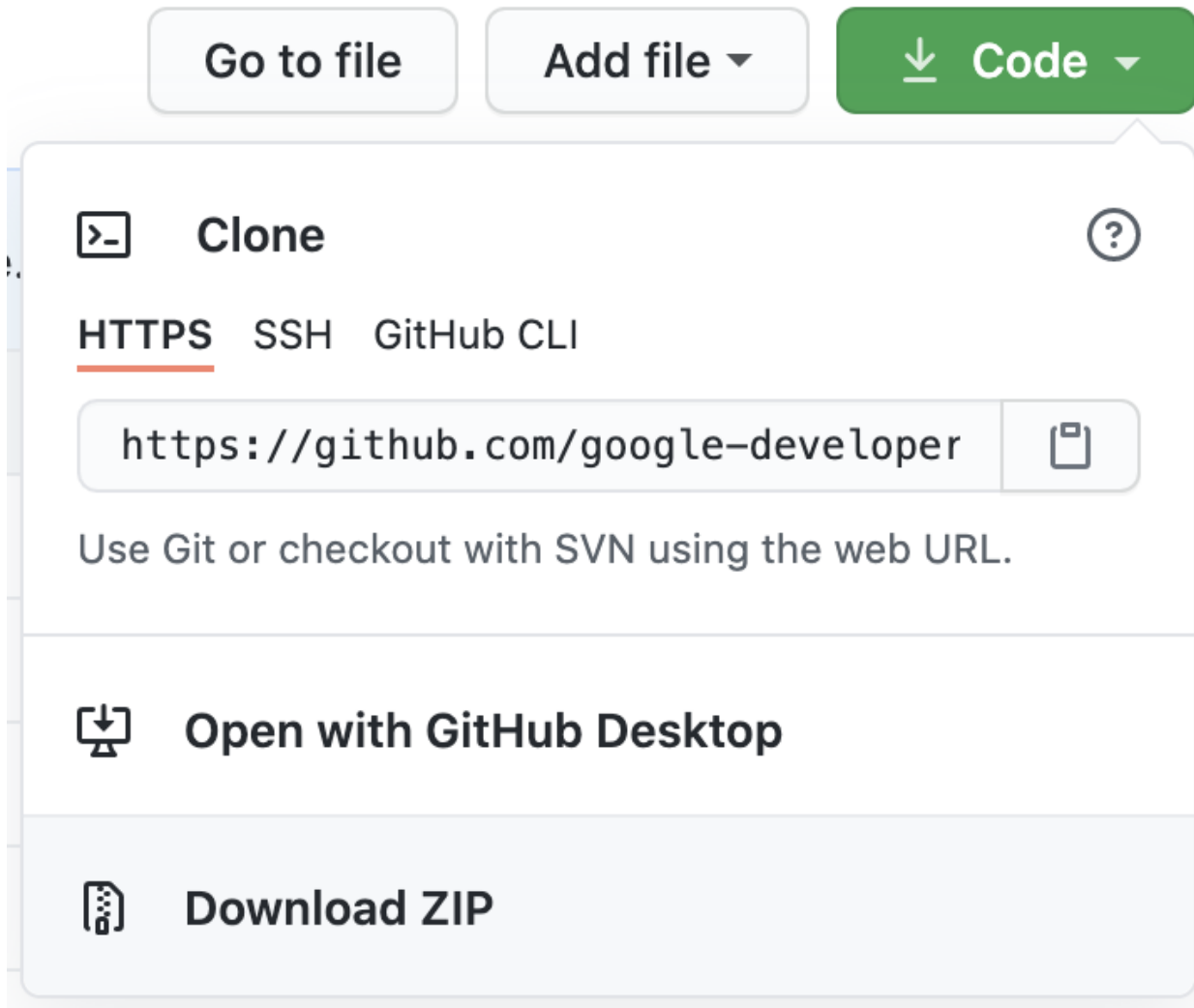
<https://github.com/google-developer-training/android-basics-kotlin-mars-photos-app/tree/starter>

Branch name: starter

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



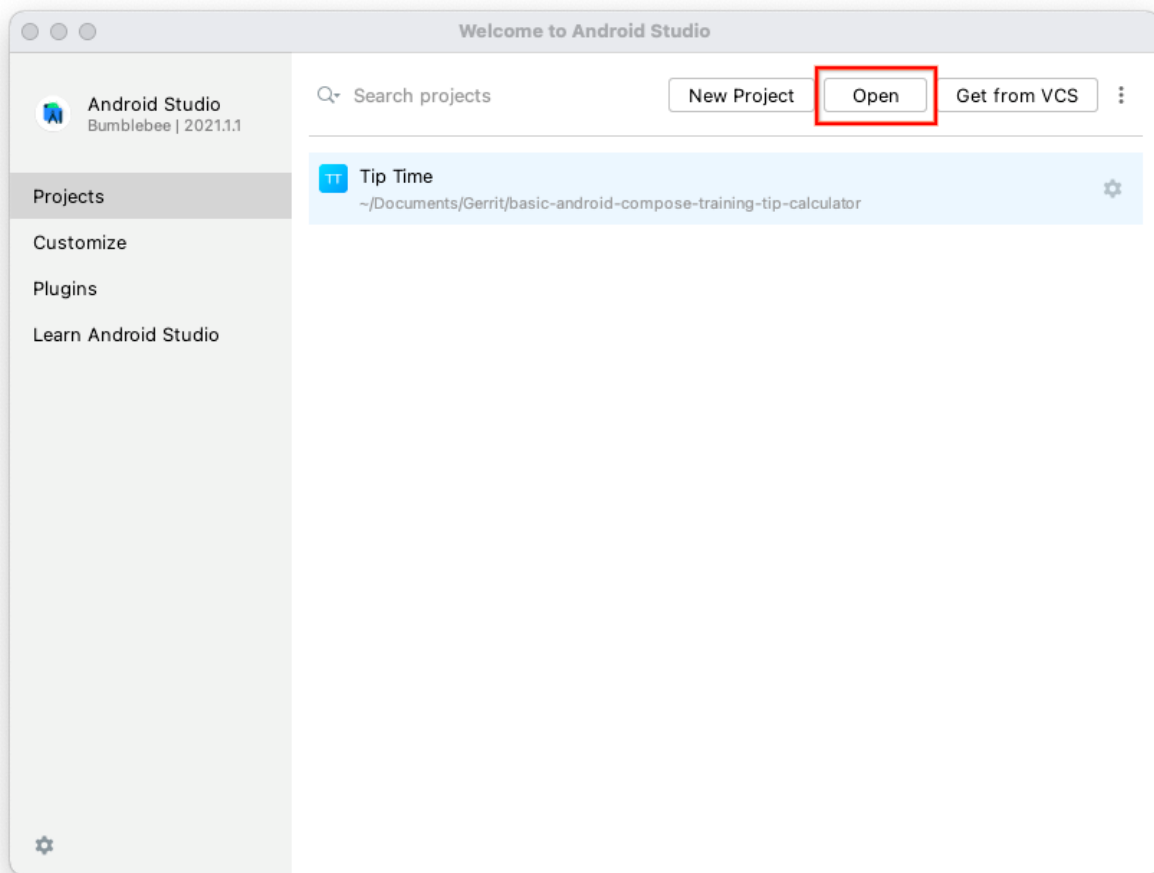
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



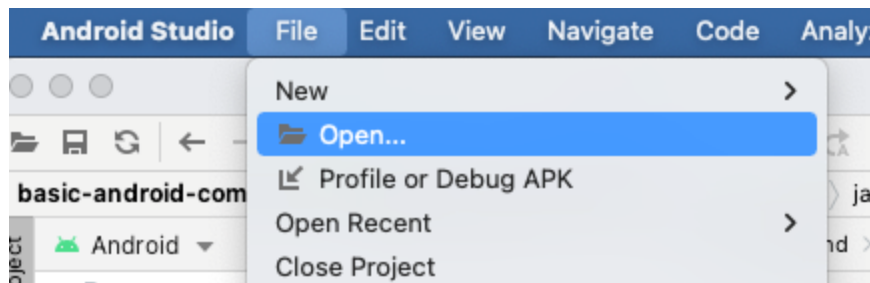
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



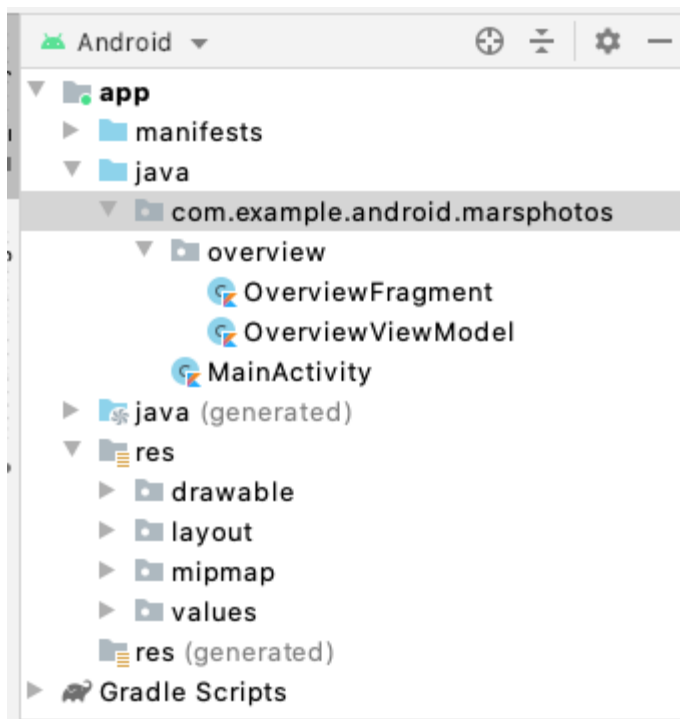
3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

Run starter code

1. Open the downloaded project in Android Studio. The folder name of the project is `android-basics-kotlin-mars-photos-app`. The starter code folder structure should look like below.
2. In the **Android** pane, expand **app** > **java**. Notice that the app has a package folder called `overview`. This is the UI layer of the app.



3. Run the app. When you compile and run the app, you should see the following screen with a placeholder text in the center. By the end of this codelab you will be updating this placeholder text with the number of photos retrieved.



4. Browse the files to understand the starter code. For layout files, you can use the **Split** option in the top right corner to see a preview of the layout and the XML at the same time.

Starter code walkthrough

In this task, you familiarize yourself with the structure of the project. Here's a walkthrough of important files and folders in the project.

`OverviewFragment:`

- This is the fragment displayed within the `MainActivity`. The placeholder text you saw in the previous step is displayed in this fragment.
- In the next codelab, this fragment will display the data received from the Mars photos backend server.
- This class holds a reference to the `OverviewViewModel` object.
- The `OverviewFragment` has an `onCreateView()` function that inflates the `fragment_overview` layout using Data Binding, sets the binding lifecycle owner to itself, and sets the `viewModel` variable in the binding object to it.
- Because the lifecycle owner is assigned, any `LiveData` used in Data Binding will automatically be observed for any changes, and the UI will be updated accordingly.

OverviewViewModel:

- This is the corresponding view model for the OverviewFragment.
- This class contains a `MutableLiveData` property named `_status` along with its backing property. Updating the value of this property, updates the placeholder text displayed on the screen.
- The `getMarsPhotos()` method updates the placeholder response. Later in the codelab you will use this to display the data fetched from the server. The goal for this codelab is to update the `status LiveData` within the `ViewModel` using real data you get from the internet.

res/layout/fragment_overview.xml:

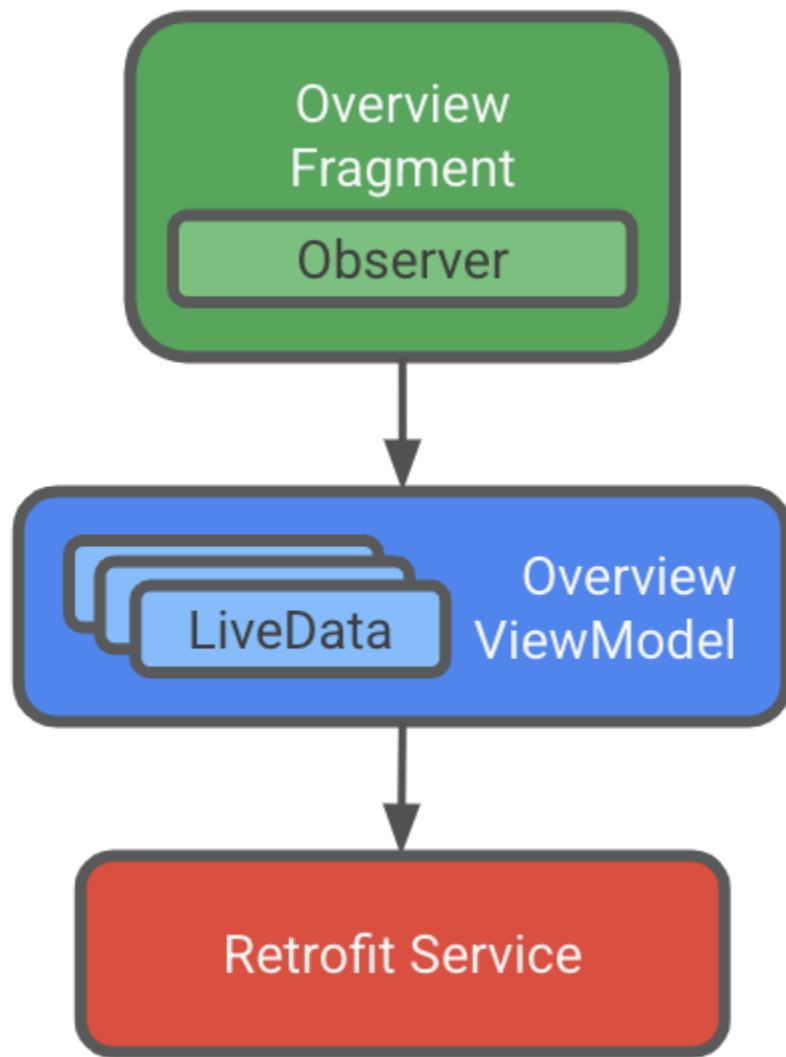
- This layout is set up to use data binding and consists of a single `TextView`.
- It declares an `OverviewViewModel` variable and then binds the `status` from the `ViewModel` to the `TextView`.

MainActivity.kt: the only task for this activity is to load the activity's layout, `activity_main`.

`layout/activity_main.xml`: this is the main activity layout with a single `FragmentContainerView` pointing to the `fragment_overview`, the overview fragment will be instantiated when the app is launched.

4. App Overview

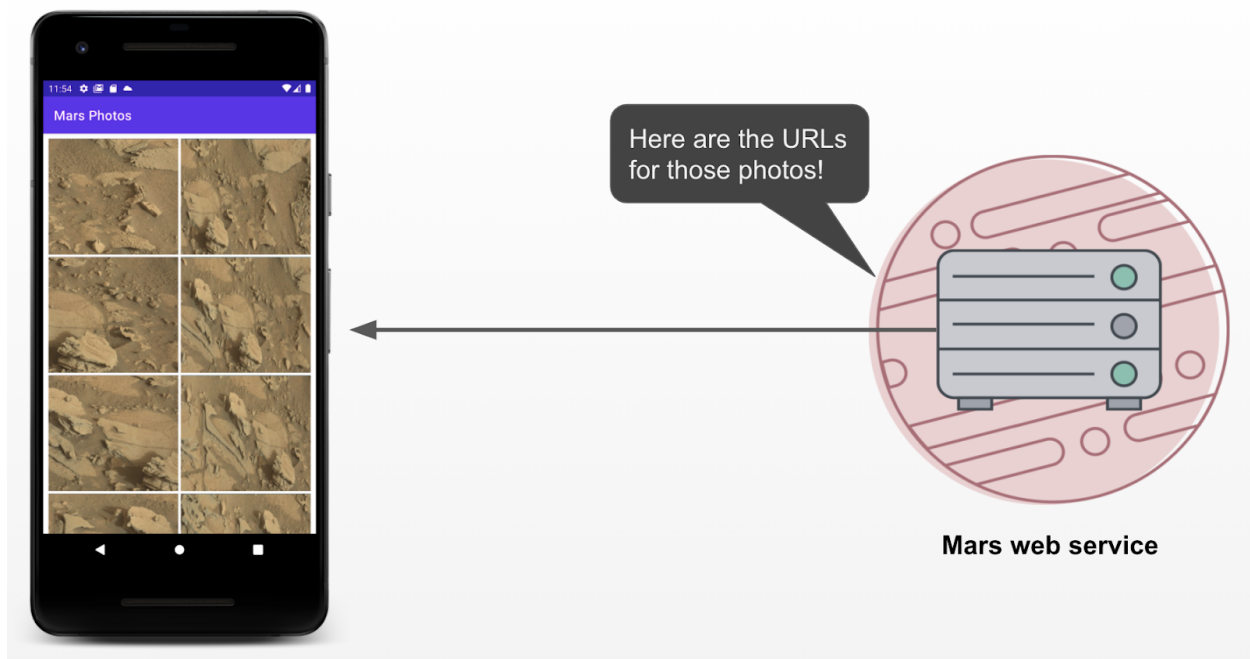
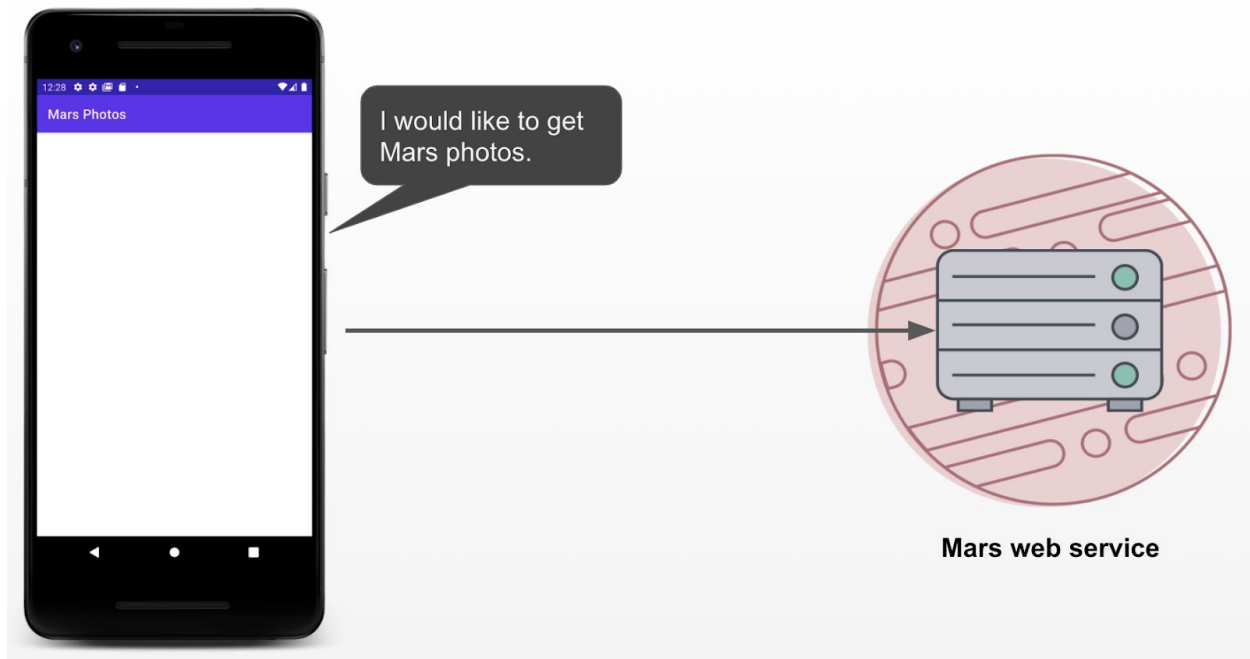
In this codelab, you create a layer for the network service that communicates with the backend server and fetch the required data. You will use a third party library to implement this, called Retrofit. You will learn more on this later. The `ViewModel` communicates directly with that network layer, the rest of the app is transparent to this implementation.



The `OverviewViewModel` is responsible for making the network call to get the Mars photos data. In the `ViewModel`, you use `LiveData` with lifecycle-aware data binding to update the app UI when the data changes.

5. Web services and Retrofit

Mars photos data is stored on a web server. To get this data into your app you need to establish a connection and communicate with the server on the internet.



Most web servers today run web services using a common stateless web architecture known as [REST](#), which stands for **RE**presentational **S**tate **T**ransfer. Web services that offer this architecture are known as RESTful services.

Requests are made to RESTful web services in a standardized way via [URIs](#). An URI (Uniform Resource Identifier) identifies a resource in the server by name, without implying its location or how to access it. For example, in the app for this lesson, you retrieve the image urls using the following server URI (This server hosts both Mars real-estate and Mars photos):

android-kotlin-fun-mars-server.appspot.com

A Uniform Resource Locator (URL) is a URI that specifies the means of acting upon or obtaining the representation of a resource, i.e. specifying both its primary access mechanism and network location.

For example:

The following URL gets a list of all available real estate properties on Mars!

<https://android-kotlin-fun-mars-server.appspot.com/realestate>

The following URL gets a list of Mars photos:

<https://android-kotlin-fun-mars-server.appspot.com/photos>

These URLs refer to a resource identified such as [/realestate](#) or [/photos](#), that is obtainable via the Hypertext Transfer Protocol (*http:*) from the network. You will be using the [/photos](#) endpoint in this codelab.

Note: The familiar web URL is actually a type of URI. Both URL and URI are used interchangeably throughout this course, depending on the API we are calling.

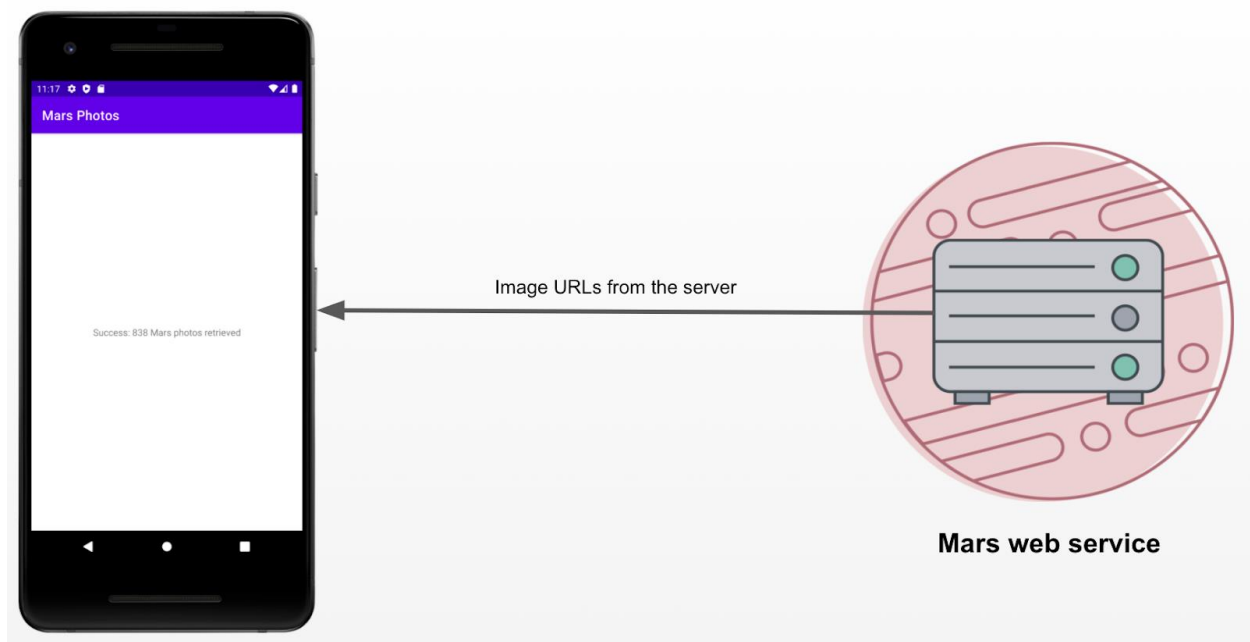
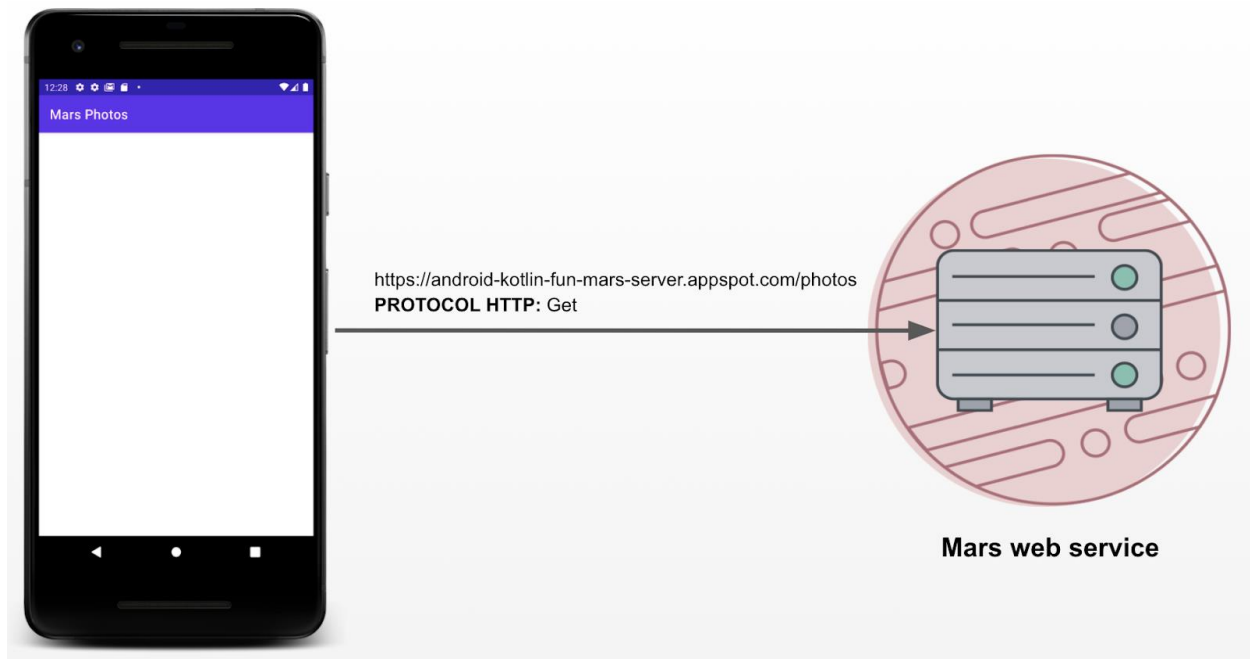
Web service request

Each web service request contains a URI, and is transferred to the server using the same HTTP protocol that's used by web browsers, like Chrome. HTTP requests contain an operation to tell the server what to do.

Common HTTP operations include:

- GET for retrieving server data
- POST or PUT for add/create/update the server with new data
- DELETE for deleting data from the server

Your app will make an HTTP GET request to the server for the Mars photos information, and then the server returns a response to our app including image urls.



The response from a web service is commonly formatted in one of the common web formats like XML or JSON – formats for representing structured data in key-value pairs. You learn more about JSON in the later task.

In this task, you establish a network connection to the server, communicate with the server, and receive a JSON response. You will be using a backend server that is already written for you. In this codelab you will use the Retrofit library, a third party library to communicate with the backend server.

External Libraries

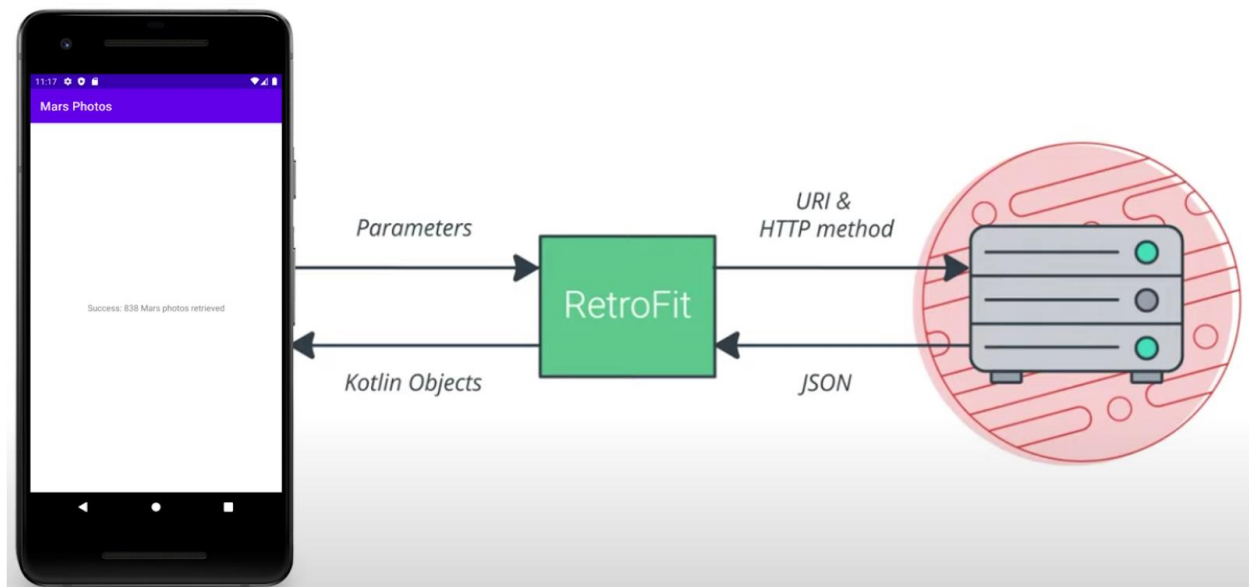
External libraries or third party libraries are like extensions to the core Android APIs. They are mostly open source, community-developed, and maintained by the collective contributions from the huge android community around the world. This enables Android developers like you to build better apps.

Warning: Using community built and maintained libraries can be a huge time-saver, but it's important to choose these libraries wisely, because your app is ultimately responsible for what the code does in these libraries.

Retrofit Library

The Retrofit library that you are going to use in this codelab to talk to the RESTful Mars web service is a good example of a well-supported and maintained library. You can tell this by looking at its [GitHub](#) page, checkout the open issues (some of them are feature requests) and closed issues. If the developers are resolving the issues and responding to the feature requests on a regular basis, then it implies that this library is well maintained and is a good candidate to use in the app. They have a [Retrofit](#) documentation page too.

Retrofit library will communicate with the backend. It creates URI's for the web service based on the parameters we pass to it. You will see more on this in later sections.



Add Retrofit dependencies

Android Gradle allows you to add external libraries to your project. In addition to the library dependency, you should also include the repository where the library is hosted. The Google libraries such as **ViewModel** and **LiveData** from the Jetpack library are hosted in the Google

repository. The majority of community libraries like Retrofit are hosted on Google and MavenCentral repositories.

1. Open the project's top-level level `build.gradle(Project: MarsPhotos)` file. Notice the repositories listed under the `repositories` block. You should see two repositories, `google()`, `mavenCentral()`.

```
repositories {  
    google()  
    mavenCentral()  
}
```

2. Open module level gradle file, `build.gradle (Module: MarsPhotos.app)`.
3. In the `dependencies` section, add these lines for the Retrofit libraries:

```
// Retrofit  
implementation "com.squareup.retrofit2:retrofit:2.9.0"  
// Retrofit with Scalar Converter  
implementation "com.squareup.retrofit2:converter-scalars:2.9.0"
```

The first dependency is for the Retrofit2 library itself, and the second dependency is for the Retrofit scalar converter. This converter enables Retrofit to return the JSON result as a `String`. The two libraries work together.

4. Click **Sync Now** to rebuild the project with the new dependencies.

Add support for Java 8 language features

Many third party libraries including Retrofit2 use Java 8 language features. The Android Gradle plugin provides built-in support for using certain Java 8 language features.

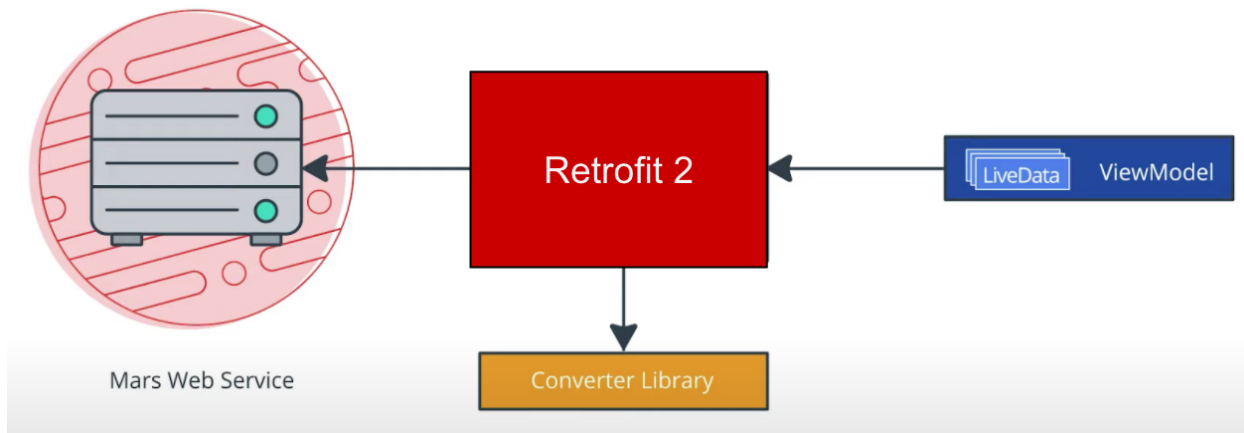
1. To use the built-in features, you need the following code in your module's `build.gradle` file. This step is already done for you, make sure the following code is present in your `build.gradle(Module: MarsPhotos.app)`.

```
android {  
    ...  
  
    compileOptions {  
        sourceCompatibility JavaVersion.VERSION_1_8  
        targetCompatibility JavaVersion.VERSION_1_8  
    }  
  
    kotlinOptions {  
        jvmTarget = '1.8'  
    }  
}
```

6. Connecting to the Internet

You will use the Retrofit library to talk to the Mars web service and display the raw JSON response as a `String`. The placeholder `TextView` will either display the returned JSON response string or a message indicating a connection error.

Retrofit creates a network API for the app based on the content from the web service. It fetches data from the web service and routes it through a separate converter library that knows how to decode the data and return it in the form of objects like `String`. Retrofit includes built-in support for popular data formats such as XML and JSON. Retrofit ultimately creates the code to call and consume this service for you, including critical details such as running the requests on background threads.



In this task, you will add a network layer to your **MarsPhotos** project that your `ViewModel` will use to communicate with the web service. You will implement the Retrofit service API, with the following steps.

- Create a network layer, the `MarsApiService` class.
- Create a Retrofit object with the base URL and the converter factory.
- Create an interface that explains how Retrofit talks to our web server.
- Create a Retrofit service and expose the instance to the api service to the rest of the app.

Implement the above steps:

1. Create a new package called `network`. In your Android project pane, right-click on the package, `com.example.android.marsphotos`. Select **New > Package**. In the popup, append **network** to the end of the suggested package name.
2. Create a new Kotlin file under the new package **network**. Name it `MarsApiService`.
3. Open `network/MarsApiService.kt`. Add the following constant for the base URL for the web service.

```
private const val BASE_URL =  
    "https://android-kotlin-fun-mars-server.appspot.com"
```

4. Just below that constant, add a Retrofit builder to build and create a Retrofit object.

```
private val retrofit = Retrofit.Builder()
```

Import `retrofit2.Retrofit`, when prompted.

5. Retrofit needs the base URI for the web service, and a converter factory to build a web services API. The converter tells Retrofit what to do with the data it gets back from the web service. In this case, you want Retrofit to fetch a JSON response from the web service, and return it as a `String`. Retrofit has a `ScalarsConverter` that supports strings and other primitive types, so you call `addConverterFactory()` on the builder with an instance of `ScalarsConverterFactory`.

```
private val retrofit = Retrofit.Builder()  
    .addConverterFactory(ScalarsConverterFactory.create())
```

Import `retrofit2.converter.scalars.ScalarsConverterFactory` when prompted.

6. Add the base URI for the web service using `baseUrl()` method. Finally, call `build()` to create the Retrofit object.

```
private val retrofit = Retrofit.Builder()  
    .addConverterFactory(ScalarsConverterFactory.create())  
    .baseUrl(BASE_URL)  
    .build()
```

7. Below the call to the Retrofit builder, define an interface called `MarsApiService`, that defines how Retrofit talks to the web server using HTTP requests.

```
interface MarsApiService {  
}
```

8. Inside the `MarsApiService` interface, add a function called `getPhotos()` to get the response string from the web service.

```
interface MarsApiService {  
    fun getPhotos()  
}
```

9. Use the `@GET` annotation to tell Retrofit that this is GET request, and specify an endpoint, for that web service method. In this case the endpoint is called `photos`. As mentioned in the previous task, you will be using [/photos](#) endpoint in this codelab.

```
interface MarsApiService {  
    @GET("photos")
```

```
    fun getPhotos()
}
```

Import `retrofit2.http.GET` when requested.

10. When the `getPhotos()` method is invoked, Retrofit appends the endpoint `photos` to the base URL (which you defined in the Retrofit builder) used to start the request. Add a return type of the function to `String`.

```
interface MarsApiService {
    @GET("photos")
    fun getPhotos(): String
}
```

Object declarations

In kotlin, [object declarations](#) are used to declare singleton objects. [Singleton pattern](#) ensures that one, and only one, instance of an object is created, has one global point of access to that object. Object declaration's initialization is thread-safe and done at first access.

Kotlin makes it easy to declare singletons. Following is an example of an object declaration and its access. Object declaration always has a name following the `object` keyword.

Example:

```
// Object declaration
object DataProviderManager {
    fun registerDataProvider(provider: DataProvider) {
        // ...
    }

    val allDataProviders: Collection<DataProvider>
    get() = // ...
}

// To refer to the object, use its name directly.
DataProviderManager.registerDataProvider(...)
```

The call to `create()` function on a Retrofit object is expensive and the app needs *only one* instance of Retrofit API service. So, you expose the service to the rest of the app using *object declaration*.

1. Outside the `MarsApiService` interface declaration, define a public object called `MarsApi` to initialize the Retrofit service. This is the public singleton object that can be accessed from the rest of the app.

```
object MarsApi {
}
```

2. Inside the `MarsApi` object declaration, add a lazily initialized retrofit object property named `retrofitService` of the type `MarsApiService`. You make this lazy initialization, to make sure it is initialized at its first usage. You will fix the error in the next steps.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {  
    }  
}
```

3. Initialize the `retrofitService` variable using the `retrofit.create()` method with the `MarsApiService` interface.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {  
        retrofit.create(MarsApiService::class.java) }  
}
```

The Retrofit setup is done! Each time your app calls `MarsApi.retrofitService`, the caller will access the same singleton Retrofit object that implements `MarsApiService` which is created on the first access. In the next task, you will use the Retrofit object you have implemented.

Note: Remember "lazy instantiation" is when object creation is purposely delayed until you actually need that object to avoid unnecessary computation or use of other computing resources. Kotlin has [first-class support](#) for lazy instantiation.

Call the web service in OverviewViewModel

In this step, you will implement the `getMarsPhotos()` method that calls the retrofit service and then handles the returned JSON string.

ViewModelScope

A [ViewModelScope](#) is the built-in coroutine scope defined for each `ViewModel` in your app. Any coroutine launched in this scope is automatically canceled if the `ViewModel` is cleared.

You will use `ViewModelScope` to launch the coroutine and make the Retrofit network call in the background.

4. In `MarsApiService`, make `getPhotos()` a suspend function. So that you can call this method from within a coroutine.

```
@GET("photos")  
suspend fun getPhotos(): String
```

5. Open `overview/OverviewViewModel`. Scroll down to the `getMarsPhotos()` method. Delete the line that sets the status response to "Set the Mars API Response here!". The method `getMarsPhotos()` should be empty now.

```
private fun getMarsPhotos() {

}
```

6. Inside `getMarsPhotos()`, launch the coroutine using `viewModelScope.launch`.

```
private fun getMarsPhotos() {
    viewModelScope.launch {
    }
}
```

Import `androidx.lifecycle.viewModelScope` and `kotlinx.coroutines.launch` when prompted.

7. Inside `viewModelScope`, use the singleton object `MarsApi`, to call the `getPhotos()` method from the `retrofitService` interface. Save the returned response in a `val` called `listResult`.

```
viewModelScope.launch {
    val listResult = MarsApi.retrofitService.getPhotos()
}
```

Import `com.example.android.marsphotos.network.MarsApi` when prompted.

8. Assign the result we just received from the backend server to the `_status.value`.

```
val listResult = MarsApi.retrofitService.getPhotos()
_status.value = listResult
```

9. Run the app, notice that the app closes immediately, it may or may not display an error popup.
10. Click the **Logcat** tab in Android Studio and note the error in the log, which starts with a line like this, "----- beginning of crash"

```
----- beginning of crash
22803-22865/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION:
OkHttp Dispatcher
    Process: com.example.android.marsphotos, PID: 22803
    java.lang.SecurityException: Permission denied (missing INTERNET
permission?)
...
```

This error message indicates the app might be missing the `INTERNET` permissions. You will resolve this by adding internet permissions to the app in the next task.

7. Add Internet permission and Exception Handling

Android Permissions

The purpose of permissions on Android is to protect the privacy of an Android user. Android apps must declare or request permissions to access sensitive user data such as contacts, call logs, and certain system features such as camera or internet.

In order for your app to access the Internet, it needs the `INTERNET` permission. Connecting to the internet introduces security concerns, which is why apps do not have internet connectivity by default. You need to explicitly declare that the app needs access to the internet. This is considered a normal permission. To learn more about Android permissions and its types please refer to the [documentation](#).

In this step, your app declares the permission(s) it requires by including `<uses-permission>` tags in the `AndroidManifest` file.

1. Open `manifests/AndroidManifest.xml`. Add this line just before the `<application>` tag:

```
<uses-permission android:name="android.permission.INTERNET" />
```

2. Compile and run the app again. If you have a working internet connection, you should see the JSON text containing data related to Mars photos. You will learn more on JSON format later in the codelab.

Mars Photos

```
[{"id": "424905", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631300503690E01_DXXX.jpg"}, {"id": "424906", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631300305227E03_DXXX.jpg"}, {"id": "424907", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631290503689E01_DXXX.jpg"}, {"id": "424908", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631290305226E03_DXXX.jpg"}, {"id": "424909", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631280503688E0B_DXXX.jpg"}, {"id": "424910", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631280305225E03_DXXX.jpg"}, {"id": "424911", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631270503687E03_DXXX.jpg"}, {"id": "424912", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631270305224E03_DXXX.jpg"}, {"id": "424913", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631260503686E03_DXXX.jpg"}, {"id": "424914", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631260305223E03_DXXX.jpg"}, {"id": "424915", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631250503685E01_DXXX.jpg"}, {"id": "424916", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631250305222E03_DXXX.jpg"}, {"id": "424917", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631240503684E03_DXXX.jpg"}, {"id": "424918", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631240305221E03_DXXX.jpg"}, {"id": "424919", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631230503683E01_DXXX.jpg"}, {"id": "424920", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631230305220E02_DXXX.jpg"}, {"id": "424921", "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631220503682E01_DXXX.jpg"}]
```

3. Tap the **Back** button in your device or emulator to close the app.
4. Put your device or emulator into airplane mode, to simulate a network connection error. Reopen the app from the recents menu, or restart the app from Android Studio.
5. Click the **Logcat** tab in Android Studio and note the fatal exception in the log, which looks like this:

```
3302-3302/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION:
main
    Process: com.example.android.marsphotos, PID: 3302
    java.net.SocketTimeoutException: timeout
...
```

This error message indicates the application tried to connect and timed out. Exceptions like this are very common in real-time. In the next step you will learn how to handle such exceptions.

Exception Handling

[Exceptions](#) are errors that can occur during the runtime(not compile time) and terminate the app abruptly without notifying the user. This can result in a poor user experience. *Exception handling* is a mechanism by which you prevent the app from terminating abruptly, and handle it in a user friendly way.

The reason for exceptions could be as simple as division by zero or an error in the network. These exceptions are similar to the `NumberFormatException` that you learned in a previous codelab.

Examples of potential issues while connecting to a server:

- The URL or URI used in the API is incorrect.
- The server is unavailable and the app could not connect to it.
- Network latency issue.
- Poor or no internet connection on the device.

These exceptions can't be caught during the compile time. You can use a `try-catch` block to handle the exception in runtime. For further learning please refer to the [documentation](#).

Example syntax for try-catch block

```
try {
    // some code that can cause an exception.
}
catch (e: SomeException) {
    // handle the exception to avoid abrupt termination.
}
```

Inside the `try` block you perform the code where you anticipate an exception, in your app this would be a network call. In the `catch` block, you will implement the code that prevents abrupt

termination of the app. If there is an exception, then the `catch` block will be executed to recover from the error instead of terminating the app abruptly.

1. Open `overview/OverviewViewModel.kt`. Scroll down to the `getMarsPhotos()` method. Inside the launch block, add a `try` block around `MarsApi` call to handle exceptions. Add `catch` block after the `try` block:

```
viewModelScope.launch {  
    try {  
        val listResult = MarsApi.retrofitService.getPhotos()  
        _status.value = listResult  
    } catch (e: Exception) {  
  
    }  
}
```

2. Inside the `catch {}` block, handle the failure response. Display the error message to the user by setting the `e.message` to the `_status.value`.

```
catch (e: Exception) {  
    _status.value = "Failure: ${e.message}"  
}
```

3. Run the app again, with the airplane mode turned on. The app does not close abruptly this time, but displays an error message instead.

4:26



Mars Photos

Failure: Unable to resolve host
"android-kotlin-fun-mars-server.appspot.com": No address
associated with hostname

4. Turn off airplane mode on your phone or emulator. Run and test your app, make sure everything is working fine and you are able to see the JSON string.

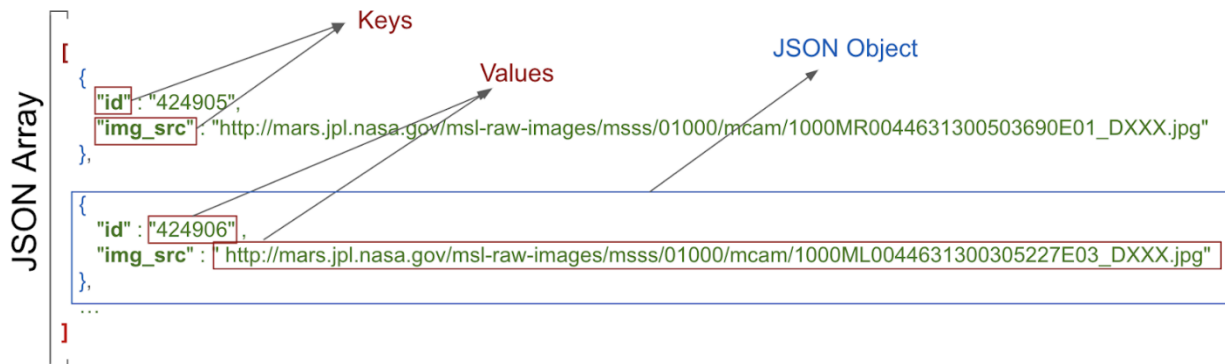
8. Parse the JSON response with Moshi

JSON

The requested data is typically formatted in one of the common data formats like XML or JSON. Each call returns structured data and your app needs to know what that structure is in order to read the data from the response.

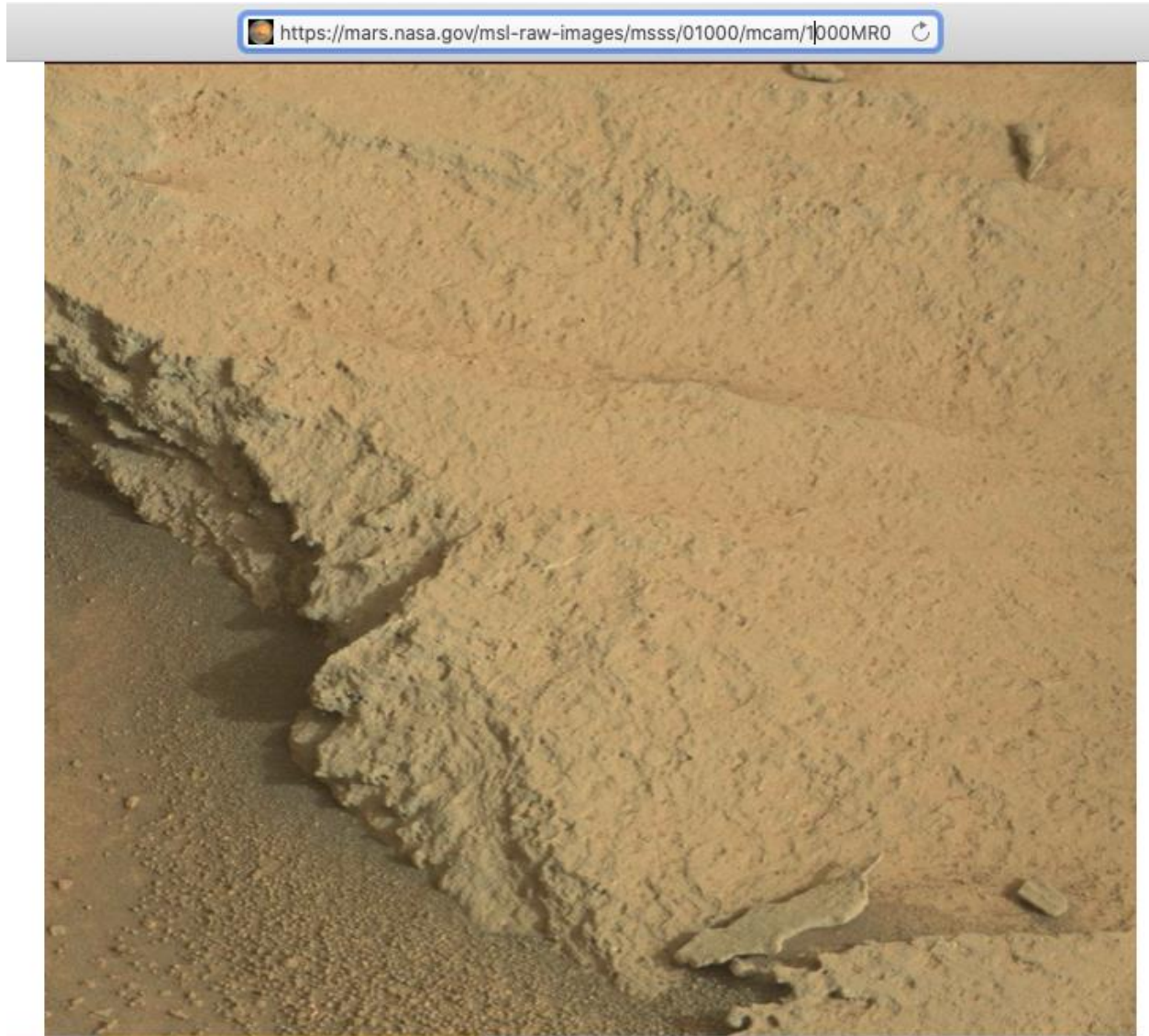
For example, in this app, you will be retrieving the data from this server: [https:// android-kotlin-fun-mars-server.appspot.com/photos](https://android-kotlin-fun-mars-server.appspot.com/photos). If you enter this URL in the browser, you will see a list of IDs and image URLs of the surface of Mars in a JSON format!

Structure of sample JSON response:



- JSON response is an array, indicated by the square brackets. The array contains JSON objects.
- The JSON objects are surrounded by curly brackets.
- Each JSON object contains a set of name-value pairs separated by a comma.
- The name and value in a pair are separated by a colon.
- Names are surrounded by quotes.
- Values can be numbers, strings, a boolean, an array, an object (JSON object), or null.

For example the `img_src` is a url, which is a string. If you paste the url into a web browser, you will see a Mars surface image.



Now you're getting a JSON response from the Mars web service, which is a great start. But what you really need are Kotlin objects, not a big JSON string. There's an external library called [Moshi](#), which is an Android JSON parser that converts a JSON string into Kotlin objects. Retrofit has a converter that works with Moshi, so it's a great library for your purposes here.

In this task, you use the Moshi library with Retrofit to parse the JSON response from the web service into useful Kotlin objects which represent Mars photos. You will change the app so that instead of displaying the raw JSON, the app displays the number of Mars photos returned.

Add Moshi library dependencies

1. Open **build.gradle (Module: app)**.
2. In the dependencies section, add the code shown below to include the Moshi dependency. This dependency adds support for the Moshi JSON library with Kotlin support.

```
// Moshi
implementation 'com.squareup.moshi:moshi-kotlin:1.13.0'
```

3. Locate the lines for the Retrofit scalar converter in the `dependencies` block and change these dependencies to use `converter-moshi`:

Replace this

```
// Retrofit
implementation "com.squareup.retrofit2:retrofit:2.9.0"
// Retrofit with scalar Converter
implementation "com.squareup.retrofit2:converter-scalars:2.9.0"
```

with this

```
// Retrofit with Moshi Converter
implementation 'com.squareup.retrofit2:converter-moshi:2.9.0'
```

4. Click **Sync Now** to rebuild the project with the new dependencies.

Note: The project may show compiler errors related to the removed Retrofit scalar dependency. You fix those in the next steps.

Implement the Mars Photos data class

A sample entry of the JSON response you get from the web service looks something like this, similar to what you have seen earlier:

```
[{
  "id": "424906",
  "img_src": "http://mars.jpl.nasa.gov/msl-raw-
images/msss/01000/mcam/1000ML0044631300305227E03_DXXX.jpg"
},
...]
```

In the example above, notice that each Mars photo entry has these JSON key and value pairs:

- `id`: the ID of the property, as a string. Since it is wrapped in " " it is of the type `String` not `Integer`.
- `img_src`: The image's URL as a string.

Moshi parses this JSON data and converts it into Kotlin objects. To do this, Moshi needs to have a Kotlin data class to store the parsed results, so in this step you will create the data class, `MarsPhoto`.

1. Right-click on the **network** package and select **New > Kotlin File/Class**.
2. In the popup, select **Class** and enter `MarsPhoto` as the name of the class. This creates a new file called `MarsPhoto.kt` in the `network` package.

3. Make `MarsPhoto` a data class by adding the `data` keyword before the class definition. Change the `{}` braces to `()` parentheses. This leaves you with an error, because data classes must have at least one property defined.

```
data class MarsPhoto(  
)
```

4. Add the following properties to the `MarsPhoto` class definition.

```
data class MarsPhoto(  
    val id: String, val img_src: String  
)
```

Notice that each of the variables in the `MarsPhoto` class corresponds to a key name in the JSON object. To match the types in our specific JSON response, you use `String` objects for all the values.

When Moshi parses the JSON, it matches the keys by name and fills the data objects with appropriate values.

@Json Annotation

Sometimes the key names in a JSON response can make confusing Kotlin properties, or may not match recommended coding style—for example, in the JSON file the `img_src` key uses an underscore, whereas Kotlin convention for properties use upper and lowercase letters ("camel case").

To use variable names in your data class that differ from the key names in the JSON response, use the `@Json` annotation. In this example, the name of the variable in the data class is `imgSrcUrl`. The variable can be mapped to the JSON attribute `img_src` using `@Json(name = "img_src")`.

5. Replace the line for the `img_src` key with the line shown below. Import `com.squareup.moshi.Json` when requested.

```
@Json(name = "img_src") val imgSrcUrl: String
```

Update MarsApiService and OverviewViewModel

In this task you will create a Moshi object using the Moshi Builder, similar to the Retrofit builder.

You will replace `ScalarsConverterFactory` with the `KotlinJsonAdapterFactory` to let Retrofit know it can use Moshi to convert the JSON response into Kotlin objects. You will then update the network API and `ViewModel` to use the Moshi object.

1. Open `network/MarsApiService.kt`. Notice the unresolved reference errors for `ScalarsConverterFactory`. This is because of the Retrofit dependency change you made in a previous step. Delete the import for `ScalarConverterFactory`. You will fix the other error soon.

Remove:

```
import retrofit2.converter.scalars.ScalarsConverterFactory
```

2. At the top of the file, just before the Retrofit builder, add the following code to create the Moshi object, similar to the Retrofit object.

```
private val moshi = Moshi.Builder()
```

Import `com.squareup.moshi.Moshi` and `com.squareup.moshi.kotlin.reflect.KotlinJsonAdapterFactory` when requested.

3. For Moshi's annotations to work properly with Kotlin, in the Moshi builder, add the `KotlinJsonAdapterFactory`, and then call `build()`.

```
private val moshi = Moshi.Builder()
    .add(KotlinJsonAdapterFactory())
    .build()
```

4. In the `retrofit` object declaration change the Retrofit builder to use the `MoshiConverterFactory` instead of the `ScalarConverterFactory`, and pass in the `moshi` instance you just created.

```
private val retrofit = Retrofit.Builder()
    .addConverterFactory(MoshiConverterFactory.create(moshi))
    .baseUrl(BASE_URL)
    .build()
```

Import `retrofit2.converter.moshi.MoshiConverterFactory` when requested.

5. Now that you have the `MoshiConverterFactory` in place, you can ask Retrofit to return a list of `MarsPhoto` objects from the JSON array instead of returning a JSON string. Update the `MarsApiService` interface to have Retrofit return a list of `MarsPhoto` objects, instead of returning `String`.

```
interface MarsApiService {
    @GET("photos")
    suspend fun getPhotos(): List<MarsPhoto>
}
```

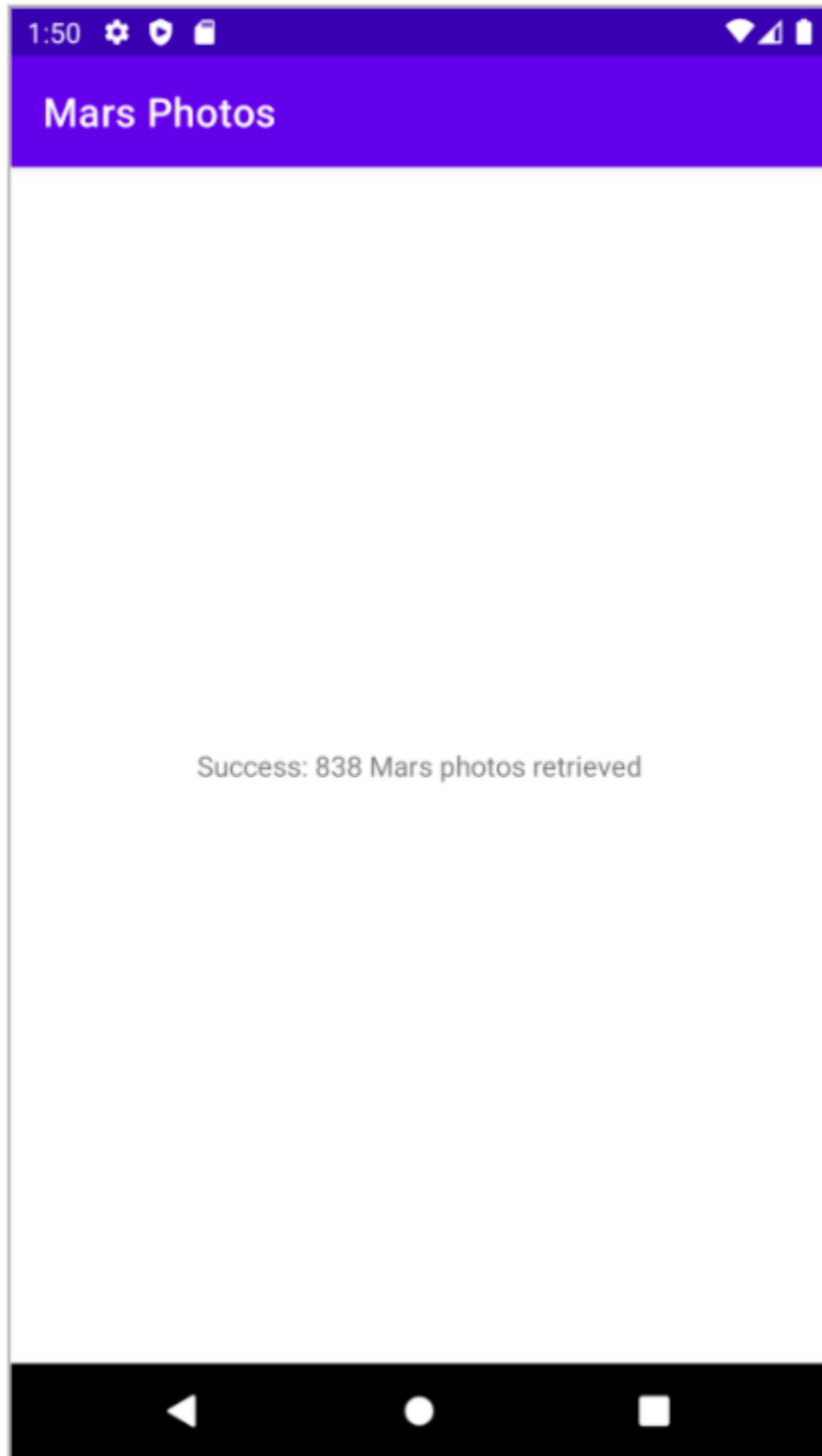
6. Do similar changes to the `viewModel`, open `OverviewViewModel.kt`. Scroll down to `getMarsPhotos()` method.

7. In the method `getMarsPhotos()`, `listResult` is a `List<MarsPhoto>` not a `String` anymore. The size of that list is the number of photos that were received and parsed. To print the number of photos retrieved update `_status.value` as follows.

```
_status.value = "Success: ${listResult.size} Mars photos retrieved"
```

Import `com.example.android.marsphotos.network.MarsPhoto` when prompted.

8. Make sure airplane mode is turned off in your device or emulator. Compile and run the app. This time the message should show the number of properties returned from the web service not a big JSON string:



Note: If your internet connection is not working, make sure that you turned off airplane mode on your device or emulator.

9. Solution code

Note: Make sure the package name in the following source code matches with the package name in your project.

build.gradle (Module: MarsPhotos.app)

These are the new dependencies to be included.

```
dependencies {  
    ...  
    // Moshi  
    implementation 'com.squareup.moshi:moshi-kotlin:1.13.0'  
  
    // Retrofit with Moshi Converter  
    implementation 'com.squareup.retrofit2:converter-moshi:2.9.0'  
  
    ...  
}
```

Manifests/AndroidManifest.xml

Add the internet permission, <uses-permission..> code from the below code snippet.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    package="com.example.android.marsphotos">  
  
    <!-- In order for our app to access the Internet, we need to define this  
    permission. -->  
    <uses-permission android:name="android.permission.INTERNET" />  
  
    <application  
        ...  
    </application>  
  
</manifest>
```

network/MarsPhoto.kt

```
package com.example.android.marsphotos.network  
  
import com.squareup.moshi.Json  
  
/**  
 * This data class defines a Mars photo which includes an ID, and the image  
 * URL.  
 * The property names of this data class are used by Moshi to match the names  
 * of values in JSON.  
 */  
data class MarsPhoto(  
    val id: String,  
    @Json(name = "img_src") val imgSrcUrl: String  
)
```

network/MarsApiService.kt

```
package com.example.android.marsphotos.network

import com.squareup.moshi.Moshi
import com.squareup.moshi.kotlin.reflect.KotlinJsonAdapterFactory
import retrofit2.Retrofit
import retrofit2.converter.moshi.MoshiConverterFactory
import retrofit2.http.GET

private const val BASE_URL =
    "https://android-kotlin-fun-mars-server.appspot.com"

/**
 * Build the Moshi object with Kotlin adapter factory that Retrofit will be
 * using.
 */
private val moshi = Moshi.Builder()
    .add(KotlinJsonAdapterFactory())
    .build()

/**
 * The Retrofit object with the Moshi converter.
 */
private val retrofit = Retrofit.Builder()
    .addConverterFactory(MoshiConverterFactory.create(moshi))
    .baseUrl(BASE_URL)
    .build()

/**
 * A public interface that exposes the [getPhotos] method
 */
interface MarsApiService {
    /**
     * Returns a [List] of [MarsPhoto] and this method can be called from a
     * Coroutine.
     * The @GET annotation indicates that the "photos" endpoint will be
     * requested with the GET
     * HTTP method
     */
    @GET("photos")
    suspend fun getPhotos() : List<MarsPhoto>
}

/**
 * A public Api object that exposes the lazy-initialized Retrofit service
 */
object MarsApi {
    val retrofitService: MarsApiService by lazy {
        retrofit.create(MarsApiService::class.java)
    }
}
```

Overview/OverviewViewModel.kt

```
package com.example.android.marsphotos.overview
```

```

import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.android.marsphotos.network.MarsApi
import kotlinx.coroutines.launch

/**
 * The [ViewModel] that is attached to the [OverviewFragment].
 */
class OverviewViewModel : ViewModel() {

    // The internal MutableLiveData that stores the status of the most recent
    request
    private val _status = MutableLiveData<String>()

    // The external immutable LiveData for the request status
    val status: LiveData<String> = _status
    /**
     * Call getMarsPhotos() on init so we can display status immediately.
     */
    init {
        getMarsPhotos()
    }

    /**
     * Gets Mars photos information from the Mars API Retrofit service and
    updates the
     * [MarsPhoto] [List] [LiveData].
     */
    private fun getMarsPhotos() {
        viewModelScope.launch {
            try {
                val listResult = MarsApi.retrofitService.getPhotos()
                _status.value = "Success: ${listResult.size} Mars photos
retrieved"
            } catch (e: Exception) {
                _status.value = "Failure: ${e.message}"
            }
        }
    }
}

```

10. Summary

REST web services

- A *web service* is software-based functionality offered over the internet that enables your app to make requests and get data back.
- Common web services use a [REST](#) architecture. Web services that offer REST architecture are known as *RESTful* services. RESTful web services are built using standard web components and protocols.

- You make a request to a REST web service in a standardized way, via URIs.
- To use a web service, an app must establish a network connection and communicate with the service. Then the app must receive and parse response data into a format the app can use.
- The [Retrofit](#) library is a client library that enables your app to make requests to a REST web service.
- Use converters to tell Retrofit what to do with data it sends to the web service and gets back from the web service. For example, the `ScalarsConverter` converter treats the web service data as a `String` or other primitive.
- To enable your app to make connections to the internet, add the `"android.permission.INTERNET"` permission in the Android manifest.

JSON parsing

- The response from a web service is often formatted in [JSON](#), a common format for representing structured data.
- A JSON object is a collection of key-value pairs.
- A collection of JSON objects is a JSON array. You get a JSON array as a response from a web service.
- The keys in a key-value pair are surrounded by quotes. The values can be numbers or strings.
- The [Moshi](#) library is an Android JSON parser that converts a JSON string into Kotlin objects. Retrofit has a converter that works with Moshi.
- Moshi matches the keys in a JSON response with properties in a data object that have the same name.
- To use a different property name for a key, annotate that property with the `@Json` annotation and the JSON key name.

[11. Learn more](#)

Android developer documentation:

- [ViewModel Overview](#)
- [LiveData Overview](#)
- [MutableLiveData](#)
- [ViewModelScope](#)

Kotlin documentation:

- [Exceptions: try, catch, finally, throw, Nothing](#)
- [Coroutines codelab](#)
- [Coroutines, official documentation](#)
- [Coroutine context and dispatchers](#)

Other:

- [Retrofit](#)
- [Moshi](#)
- [Moshi with Retrofit in Kotlin](#)